

Contents

SISTEMSKO PROJEKTOVANJE.....	2
Attention mehanizam	2
Princip rada Multi-Head Attention mehanizma	3
Analiza profiler rezultata	6
Ekstrakcija i C++ modeliranje	8
Bitska analiza	10
Verifikacija i virtuelna platforma	11
Verifikacije modela i analiza performansi	15
Implementacija hardvera (HLS)	16

SISTEMSKO PROJEKTOVANJE

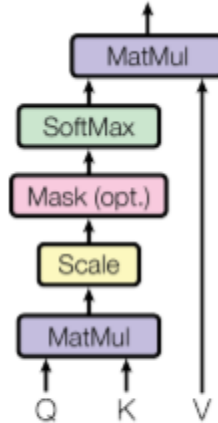
Attention mehanizam

-Mehanizam, koji je ključna komponenta Transformer arhitekture, igra glavnu ulogu u poboljšanju mogućnosti različitih modela (u našem slučaju whisperAI model) da se istovremeno fokusiraju na više drugačijih sekvenci ulaznih signala, posebno je veći doprinos u zadacima poput mašinskog učenja, generisanje teksta i još mnogo toga.
-Ulaz se može predstaviti kao skup matrica Q (Query), K (Key), i V (Value):

- **Q** – pitanje:”na koje delove audio snimka trebam da obratim pažnju u datom trenutku“?
- **K** – predstavlja index svih segmenata zvučnog fajla. Na osnovu njih model poredi svoj upit (Q) sa dostupnih informacijama kako bi pronašao najbolje poklapanje,
- **V** – stvarni zvučni podatak. Kada se pitanje (Q) i naslov (K) poklope, model uzima vrednost iz ove matrice kako bi generisao izlaz.

-Mehanizam samopažnje (eng. Self-Attention) se definiše pomoću formule:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$



Slika 1. Arhitektura mehanizma samopažnje

-Matrice Q, K i V izvede se iz istog izvora (izlaz prethodnog sloja). Ovo omogućava modelu da analizira odnose između različitih delova sekvence. Nakon množenja matrica Q, K i V, dobijaju se sirovi podaci na koje se primenjuje **softmax funkcija**, koja osigurava da suma svih težina bude tačno 1. Ova funkcija pretvara sirove brojeve u verovatnoće, govoreći modelu da obrati pažnju na neku specifičnu reč u rečenici. Na kraju se rezultat softmax funkcije množi sa sa vrednostima V, gde se na izlazu dobija **težinska suma**.

Princip rada Multi-Head Attention mehanizma

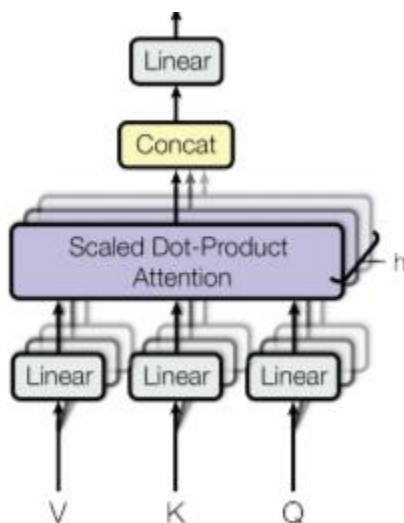
-Kod mehanizma pažnje sa više glava (eng. Multi-Head Attention), ulaz se deli na više glava, što omogućava modelu da uhvati različite vrste odnosa u podacima. Umesto korišćenja samo jednog većeg skupa prethodno spomenutih matrica, ulazne reprezentacije (embeddings) se projektuju u više skupova (glava), pri čemu svaka glava ima sopstvene matrice. Proces se odvija u sledećim koracima:

- **Linearna transformacija:** ulaz X se projektuje u više potprostora manjih dimenzija koristeći različite težinske matrice.

$$Q_i = XW_i^Q, K_i = XW_i^K, V_i = XW_i^V$$

gde i označava index glave.

- **Nezavisno izračunavanje pažnje:** svaka glava nezavisno računa svoju samopažnju koristeći formulu skaliranog skalarnog proizvoda (eng. Scaled Dot-Product Attention).
- **Nadovezivanje:** izlazi svih pojedinačnih glava se spajaju u jednu matricu.
- **Konačna linearna transformacija:** na kraju se primenjuje konačna težinska matrica kako bi se spojeni izlaz transformisao u odgovarajući vektor.



Slika 2. Arhitektura mehanizma pažnje sa više glava

-Matematički, mehanizam pažnje sa više glava se može prikazati kao:

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \text{head}_2, \dots, \text{head}_h)W^O$$

odakle sledi:

$$\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$$

- W^O je finalna težinska matrica koja projektuje spojeni izlaz nazad u dimenzije koje zahteva model.

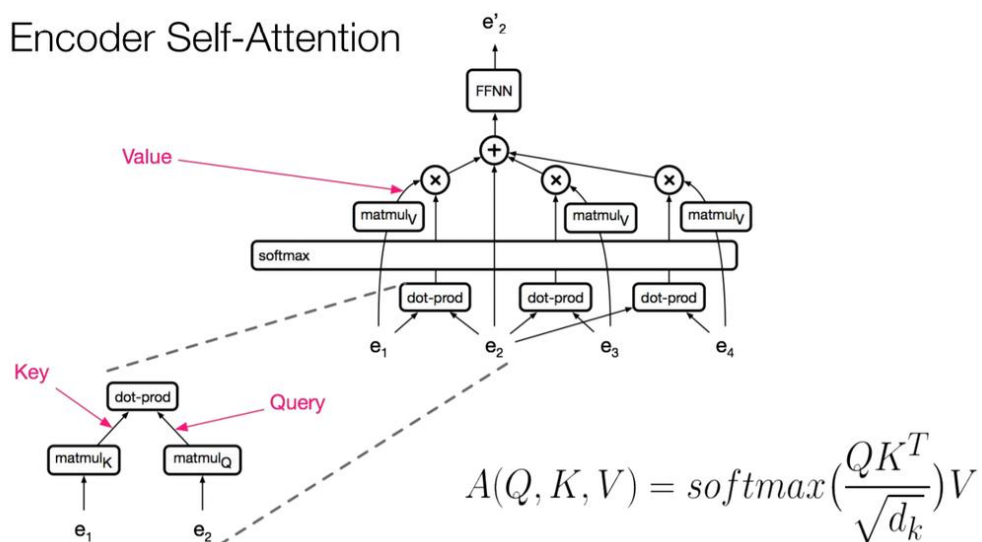
-Multi-Head Attention pruža nekoliko prednosti:

- različite glave se fokusiraju na različite aspekte ulaza,
- radeći paralelno, višestruke glave omogućavaju bolje učenje zavisnosti unutar podataka,
- model se ne oslanja samo na jedan obrazac, čime se smanjuje rizik od preprilagođavanja (overfitting).

-Mehanizam sa više glava se koristi na nekoliko mesta unutar Transformer modela:

- **Samopažnja enkodera (Encoder Self-Attention):** Ovo omogućava enkoderu da nauči kontekstulane odnose unutar ulazne sekvence. Svaka reč (ili Token) u ulazu obraća pažnju na svaku drugu reč, pomagajući modelu da razume zavisnost bez obzira na njihovu međusobnu udaljenost u sekvenci.

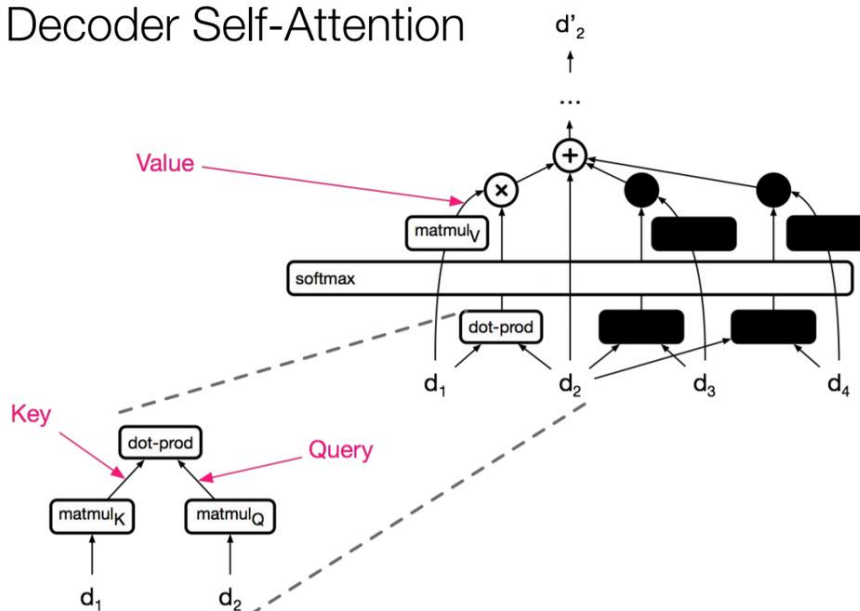
Encoder Self-Attention



Slika 3. Arhitektura enkoder samopažnje

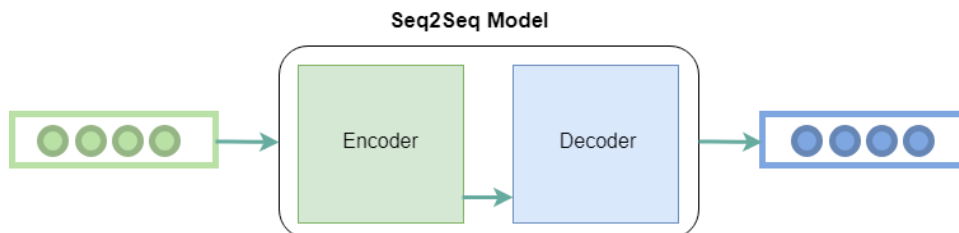
- **Samopažnja dekodera (Decoder Self-Attention):** Osigurava da svaka pozicija u izlaznoj sekvenci može obraćati pažnju samo na prethodne pozicije. Ovo se postiže korišćenjem mehanizma maskiranja (eng. masking mechanism), koji sprečava dekodera da vidi buduće tokene tokom procesa obuke. Ovakav pristup omogućava generisanje sekvenci na autoregresivan način, dok se model fokusira isključivo na relevantne delove onog što je do tog trenutka generisano.

Decoder Self-Attention



Slika 4. Arhitektura dekodera samopažnje

- **Enkodersko-dekoderska pažnja (Encoder-Decoder Attention):** Ovaj sloj omogućava dekoderu da obraća pažnju na izlaz enkodera. Pomaže dekoderu da se uskladi i fokusira na odgovarajuće ulazne tokene prilikom generisanja svakog pojedinačnog izlaznog tokena. Upravo ovaj mehanizam omogućava uspešno izvršavanje zadataka tipa sekvenca-u-sekvencu (eng. sequence-to-sequence), kao što je mašinsko učenje.



Slika 5. Enkodersko-dekoderska pažnja

Analiza profiler rezultata

-Pre same faze ekstrakcije i modelovanje u C++ jeziku, izvršeno je profilisanje originalnog Python koda kako bi se precizno identifikovali delovi modela koji najviše opterećuju procesor.

-Analizom hijerarhije poziva, uočeno je da ukupno vreme transkripcije u testiranoj aplikaciji (final_app.py) iznosi 7.7s. Unutar tog vremena, dominantnu ulogu imaju funkcije za generisanje tokena:

- **WhisperForConditionalGeneration.generate** i
- **GenerationMixin.generate**

-Ove funkcije se pozivaju u petlji za svaki novi generisani token. One zapravo služe da u svakoj iteraciji pokreću složene proračune unutar dekodera, gde se nalazi **Multi-Head attention (MHA)** mehanizam.

-Da bi smo lakše razumeli šta konkretno troši vreme unutar pomenutih funkcija, korišćen je detaljan prikaz operacija.

Area Selection		Slices	Pivot Table		Slice Flamegraph		
Name	Wall duration ↓		Self duration		Avg Wall duration	Occurrences	
	Σ	2.641s	Σ	969.1ms		Σ	39575
model_inference		969.1ms		128.5ms	969.1ms		1
aten::linear		477.0ms		9.899ms	314.3μs		1518
aten::addmm		292.3ms		274.7ms	225.5μs		1296
aten::bmm		192.7ms		192.0ms	263.2μs		732
aten::matmul		159.1ms		2.290ms	716.5μs		222
aten::mm		155.5ms		155.3ms	700.4μs		222
aten::softmax		62.43ms		952.5μs	170.6μs		366
aten::softmax		61.48ms		61.48ms	168.0μs		366
aten::copy_		29.08ms		29.08ms	16.34μs		1779
aten::layer_norm		18.84ms		2.560ms	32.31μs		583
aten::clone		16.62ms		481.4μs	339.2μs		49
aten::native_layer_norm		16.28ms		8.006ms	27.92μs		583
aten::gelu		15.82ms		15.82ms	84.15μs		188
aten::contiguous		14.64ms		156.9μs	340.6μs		43
aten::conv1d		13.16ms		18.12μs	6.579ms		2
aten::convolution		13.14ms		146.2μs	6.570ms		2
aten::_convolution		12.99ms		63.20μs	6.497ms		2
aten::mkldnn_convolution		12.89ms		12.82ms	6.445ms		2
aten::add		12.20ms		12.20ms	19.90μs		613
aten::transpose		11.05ms		7.935ms	3.683μs		3000
aten::view		10.64ms		10.64ms	1.594μs		6675
aten::reshape		10.26ms		4.204ms	3.834μs		2677
aten::t		8.713ms		3.592ms	5.739μs		1518
aten::mul		7.756ms		4.193ms	16.97μs		457
aten::to		6.080ms		1.764ms	14.14μs		430
aten::as_strided		4.543ms		4.543ms	948ns		4788
aten::_to_copy		4.317ms		2.584ms	10.87μs		397
aten::expand		3.806ms		2.820ms	2.806μs		1356
aten::eq		3.638ms		1.479ms	117.4μs		31
aten::empty		3.228ms		3.228ms	1.674μs		1928
aten::cat		3.149ms		2.770ms	8.331μs		378

Slika 6. Potrebna vremena izvršavanja određenih matematičkih operacija

-Iz ove tabele se jasno vidi da su glavna uska grla (bottlenecks) upravo matematičke operacije koje čine srž Multi-Head Attention mehanizma:

- Operacije **aten::linear** (477 ms), **aten::matmul** (159.1 ms) i **aten::bmm** (192.7ms) zajedno čine ubedljivo najveći deo vremena. U kontekstu projekta, ove operacije direktno odgovaraju linearnim transformacijama za dobijanje Q, K i V matrica, kao i samom procesu izračunavanja težine pažnje ($Q \times K^T$).
- Operacija **aten::softmax** (zajedno sa internim varijacijama) troši preko 120ms. Ovo je ključna komponenta MHA koja osigurava da suma težina pažnje bude 1.
- Operacije poput **transpose**, **reshape** i **view** su neophodne za deljenje i spajanje glava (Split/Merge Heads) unutar MHA modula.

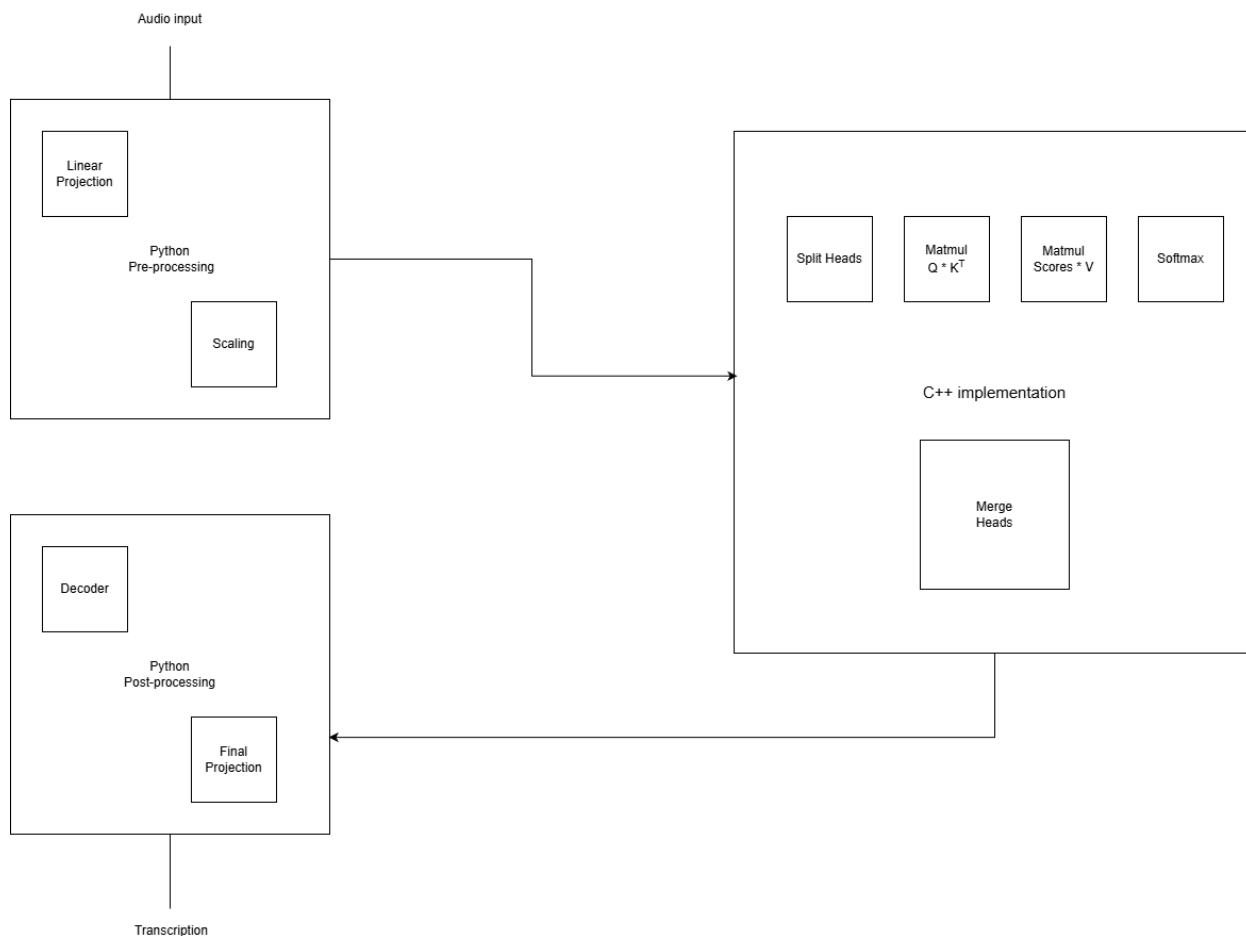
-Ovi podaci nam pružaju informaciju koliko standardna Python implementacija troši vremena na bazične matrične operacije. To nam dozvoljava da kompletan MHA mehanizam sktrahujemo i implementiramo.

Ekstrakcija i C++ modeliranje

-Prva faza projekta jeste ekstrakcija mehanizma pažnje sa više glava iz originalnog WhisperAI modela, koji je implementiran u jeziku Python. Primarni cilj ove faze bio je prelazak sa softverskog algoritma na implementaciju u jeziku C++, koja je bolja za kasniju hardversku realizaciju. Ključni korak u verifikaciji je razvoj **pybind_wrapper.cpp** fajla. Ovaj model predstavlja direktnu implementaciju algoritma prilagođenu za kasniji prelazak na hardver, ali sa integrisanom podrškom za Python okruženje. Glavni izazov je bio osigurati da ovaj C++ kod imitira ponašanje originalnog modela. Proces verifikacije je tekao na sledeći način:

- originalni Python sloj je isključen,
- umesto njega je integrisan naš **pybind_wrapper.cpp** fajl (preko Pybind11),
- izvršeno je testiranje novog modela istim audio fajlovima,

-Nakon testiranja, izlaz je bio indentičan izlazu originalne implementacije, i na taj način je utvrđena tačnoost našeg modela. Princip povezivanja i koraci izvršavanja je prikazano pomoću blok šeme.



Slika 6. blok dijagram komunikacije između *final_app.py* i *pybind_wrapper.cpp*

Detaljan opis implementacije

-Nakon definisanja opšte ahitekture, implementacija je podeljena na softverski (final_app.py) i hardverski (pybind_wrapper.cpp) deo.

-Softverski deo:

- **Linearna projekcija (Linear Projection):** Ulazni tenzori se projektuju u Q, K i V prostore koristeći predefinisane težine modela.
- **Skaliranje:** Matrica Q se skalira faktorom $\frac{1}{\sqrt{d_k}}$ direktno u Python-u kako bi se smanjila kompleksnost ulaza hardverskog modula:

```
scale_factor = 1.0 / np.sqrt(self.head_dim)
q_list = Q.detach().cpu().numpy() * scale_factor
```

- **Post-processing(dekoder i finalna projekcija):** Rezultati, dobijeni iz hardvera, se dekoduju i zatim se primenjuje finalna projekcija:

```
attn_output = self.out_proj(attn_output)
```

-Hardverski deo:

- **Deljenje glava (Split Heads):** Ulazne matrice se dele na nezavisne glave pažnje.
- **Množenje matrica (Q * K^T):** Za svaku glavu pojedinačno se vrši množenje matrica Q i K.
- **Softmax:** Nad rezultatom prvog množenja primenjuje se softmax.
- **Množenje matrica (Scores * V):** Rezultat softmax Scores se pritom množi matricom V.

```
for (int h = 0; h < num_heads; ++h) {
    Matrix scores = matmul_transpose(q_heads[h], k_heads[h]);
    Matrix probs = softmax_internal(scores);
    attn_output_heads[h] = matmul_standard(probs, v_heads[h]);
}
```

- **Spajanje glava (Merge Heads):** Rezultat svih glava pojedinačno se u finalnom koraku spaja u jednu veliku matricu, i šalje softveru.

-Uspešnost oba dela rezultuje u ispravnoj transkripciji.

Bitska analiza

-Nakon što smo potvrdili da je naša implementacija hardvera funkcionalna, sledeća faza projekta jeste prelazak na aritmetiku u fiksnom zarezu (Fixed-point) kako bismo smanjili potrošnju resursa na FPGA čipu, uz zadržavanje preciznosti.

-Za svaku operaciju u mehanizmu pažnje (npr. softmax), analizirali smo opseg vrednosti pomoću naše funkcije **analyze_bits**. Format broja je definisan kao $Q_{I,F}$ gde je:

- I (Integer bits) – broj bitova za celobrojni deo, određen na osnovu formule: $I = \lceil \log_2(\max_abs_val) \rceil + 1$ (1 bit je potreban za znak broja)
- F (Fractured bits) – broj bitova za izlomljeni deo, određen na osnovu najmanje vrednosti koja utiče na rezultat.

-Na osnovu podataka dobijenih audio signalom, primer rezultata funkcije za analizu broja bitova su:

- Analiza: Ulaz Q | Opseg: (6.89179e-08 ... 1.24322) | Potreban format: Q2.20 (Ukupno: 22 bita)
- Analiza: Head 0 Scores | Opseg: (4.19939e-06 ... 12.7354) | Potreban format: Q5.18 (Ukupno: 23 bita)
- Analiza: Head 4 Scores(Softmax) | Opseg: (1.00019e-10 ... 0.988285) | Potreban format: Q2.20 (Ukupno: 22 bita)...

-Ovi rezultati nam pokazuju da je opseg izlomljenog dela dosta važniji za očuvanje tačnosti modela.

-Ova vrsta analize je jako bitna pri prelasku sa **double** i **float** na **sc_fixed** ili **ap_fixed**. Na primer, za Ulaz Q, na osnovu analize vidimo da nam je potrebno 22 bita, 2 za celi, i 20 za izlomljeni deo. Tako da, to možemo definisati kao **using DATA_T = sc_fixed<22, 2>**, gde je od 22 bita, 2 celobrojni deo, a 20 izlomljeni deo, što spada u opseg koji je nama potreban.

-Primer pozivanja funkcije u kodu:

```
Matrix scores = matmul_transpone(q_heads[h], k_heads[h]);  
analyze_bits("string koji zelimo", scores);
```

-Sve deklaracije, poput ovog primera, su definisane unutar **datatypes.h** fajla.

-Kompletna prikaz bitske analize nalazi se unutar **bitska_analiza.txt** fajla.

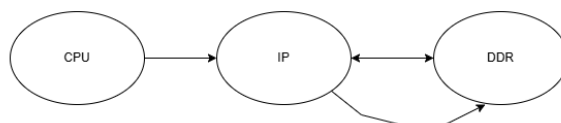
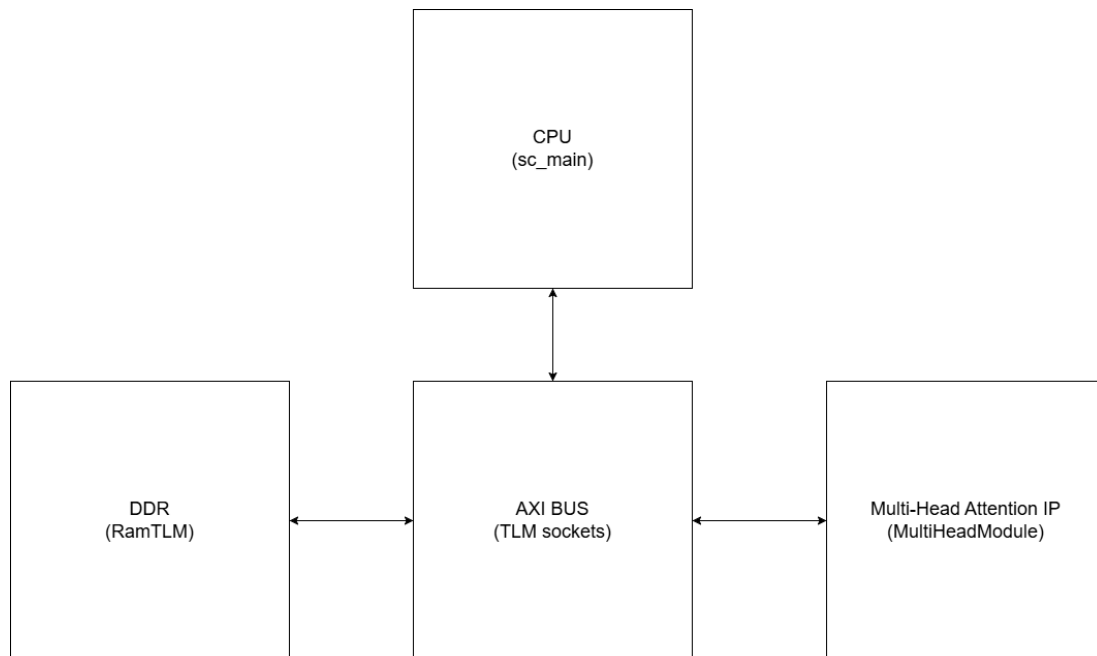
Verifikacija i virtuelna platforma

-Kada konačno imamo sve informacije i fajlove koji su nam potrebni, možemo razviti virtuelno okruženje kao **TLM (Transaction Level Modeling)** komponentu. Dizajnirana je tako da simulira realno hardversko okruženje pri same implementacije na FPGA čipu.

-Ključne komponente platforme su: model koji simulira CPU za slanje podataka preko AXI magistrale, glavnu memoriju predstavlja DDR memorija gde se nalaze Q, K i V matrice, i gde se upisuje rezultat matrica Y. Takođe, modelovali smo i magistralu koja povezuje CPU, memoriju i C++ algoritam.

-Magistrale za komunikaciju između modula, su modelovane korišćenjem **TLM-2.0** standarda. Platforma koristi **Loosely Timed (LT)** mehanizam sa eksplicitnim modelovanjem latency pristupa memoriji (10ns po transakciji), što omogućava grubu procenu performansi sistema.

-Zamena algoritma ili proširenje memorije ne zahteva promenu interfejsa, jer su svi moduli povezani preko standardizovanih **TLM socket-a**.



Slika 7. Arhitektura SystemC VP TLM 2.0

-Opis komponenti virtuelne platforme:

- **CPU:** Predstavlja glavni procesor koji upravlja radom hardvera. On inicira transakcije slanjem komandnih signala i parametara (poput adresa matrica u memoriji) ka IP modelu.
- **AXI BUS:** Centralni deo sistema koji omogućava komunikaciju između master (CPU, IP) i slave (DDR, IP) komponenti. Modelovano je pomoću **TLM-2.0**, što omogućava brzu simulaciju bez potrebe za modelovanje svakog pojedinačnog signala.
- **Multi-Head Attention IP:** Glavna komponenta platforme koja implementira C++ algoritam. Može da radi u dve uloge: kada prima instrukcije od CPU-a, i kada samostalno pristupa memoriji radi čitanja ili upisa podataka.
- **DDR memorija:** Simulira memorijski prostor koji čuva ulazne podatke (Q, K, V) i rezerviše prostor za izlazni rezultat (Y).

-Nakon inicijalizacije, rad virtuelne platforme se odvija na sledeći način:

- CPU šalje transakcije preko AXI magistrale kako bi podesio registre unutar IP modula,
- IP šalje zahtev DDR memoriji za dobavljanje redova matrica Q, K i V,
- unutar IP modela izvršava se niz operacija nad datim podacima iz memorije,
- nakon izvršavanja logike unutar IP-a, rezultat se šalje DDR memoriji gde se upisuje,
- CPU se obaveštava da je finalna matrica kreirana, nakon čega procesor može pristupiti tim podacima.

-Unutar **hardware.h** i **hardware.cpp** fajla nalazi se kompletna deklaracija funkcija, promenljivih i logika koju hardver izvršava. Izuzev memorije, ona je napisana u posebnim fajlovima pod nazivom **ram_tlm.h** i **ram_tlm.cpp**.

-Host fajl, pokretanje/prekidanje simulacije, učitavanje podataka, ispisi, i upravljanje simulacionim vremenom, se sve izvršava unutar **testbench.cpp**.

-Kako bi se olakšalo praćenje toka simulacije, tabela 1 daje pregled ključnih funkcija na praktičnom primeru:

Modul	Funkcija	Klasa
Host (CPU) (testbench.cpp)	load_matrix_from_file(...)	Na početku (sc_main), ova funkcija čita .txt fajlove sa diska i upisuje ih direktno u RamTLM koristeći direct_write , simulirajući punjenje memorije.
Kontroler (testbench.cpp)	SimulationControl::monitor()	Proces koji neprekidno nadgleda signal done . Kada MultiHeadModule završi posao, ova funkcija poziva sc_stop() i gasi simulaciju.
Hardver (hardware.cpp)	mem_read(addr)	Pomoćna funkcija unutar MatrixMultiplier klase koja pakuje adresu u generičku transakciju i šalje je memoriji.
DDR Memorija (ram_tlm.cpp)	b_transport(...)	Simulira kašnjenje. Kada primi zahtev TLM_READ/WRITE , kopira podatke memcpy i dodaje kašnjenje od 10ns delay = sc_time(10, SC_NS) .
Logika množenja (hardware.cpp)	MatrixMultiplier::multiply()	Glavni SC_THREAD proces koji izvršava množenje matrica. Sadrži tri ugnježdene petlje koje čitaju podatke, množe ih i upisuju rezultat.
Upravljanje glavama (hardware.h)	SingleHeadModule	Modul koji diriguje rad. On instancira MatrixMultiplier i prosleđuje mu ispravne adrese za Q, K i V matrice za svaku glavu.

Tabela 1. Mapiranje sistemskih funkcija i modula

-U okviru virtuelne platforme, hardverska logika nije implementirana kao jedan blok, već je podeljena radi lakše verifikacije, prateći strukturu definisanu u hardware.h:

- **MultiHeadModule (Top Level):** Ovo je glavni blok koji simulira IP. On u sebi sadrži instancu **SingleHeadModule** i povezuje njihove signale sa taktom (clk).

- **SingleHeadModule (Kontrola):** Ovaj modul je zadužen za upravljanje adresama. Na osnovu baznih adresa u memoriji (gde počinju Q, K i V), on računa ofsete za svaku specifičnu glavu mehanizma pažnje i pokreće proces množenja.
- **MatrixMultiplier (Aritmetička jedinica):** Glavna logika množenja, i nalazi se u funkciji multiply(). To je **SC_THREAD** proces koji simulira ponašanje hardverskog množača matrica. Koristi funkciju **mem_read()** da povuče elemente matrica X i W iz **RamTLM** memorije
- Nakon toga, izvršava se standardni algoritam množenja (red x kolona) unutar ugnježđenih pelji. Rezultat se upisuje nazad u memoriju pozivom **mem_write()**.

-Kako bi se simulirao realan rad sistema, **virtuelna RAM** memorija je logički podeljena na segmente. Svaka matrica ima svoju baznu adresu definisanu u **testbench.cpp** fajlu. Ovi ofseti su odabrani tako da se osigura dovoljno prostora za smeštanje matrica maksimalnih dimenzija (1500 x 512), bez rizika od preklapanja podataka.

Podaci	Adresa u memoriji (Hex)	Offset
Q	0x0000000	0 MB
K	0x0A00000	+10 MB
V	0x1400000	+20 MB
W	0x1E00000	+30 MB
Bias Vektor (b)	0x2000000	+32 MB
Y	0x2800000	+40 MB

Tabela 2. Mapa adresa u virtuelnoj memoriji

-Radi lakšeg pristupa hardvera memoriji, kreiran je standardizovan objekat **tlm_generic_payload** koji simulira paket na magistrali. Svaka transakcija u **hardware.cpp** sadrži tri ključne informacije:

- **set_command:** definiše smer prenosa – **TLM_READ_COMMAND** (čitanje iz RAM-a) ili **TLM_WRITE_COMMAND** (upis u RAM).
- **set_address:** precizna lokacija podataka u memoriji.
- **set_data_ptr:** adresa promenljive u koju će se smestiti pročitana vrednost ili iz koje se uzima vrednost za upis.

-Ovaj mehanizam omogućava da se kompletna složenost prenosa podataka apstrahuje u jedan C++ objekat, čineći kod preglednijim i lakšim.

Verifikacije modela i analiza performansi

-Za potrebe estimacije performansi pre hardverske implementacije, korišćen je TLM 2.0 LT (Loosely Timed) model.

-Ovaj pristup omogućava simulaciju vremena izvršavanja. Ukupno simulaciono vreme predstavlja zbir vremena potrebnog za prenos podataka između IP-a i memorije (Interconnect + Memory Access) modelovano kroz **b_transport** funkciju, i samo izvršavanje matematičkih operacija. Vreme obrade podataka unutar DSP blokova modelovano je pomoću wait() instrukcije unutar petlji, bazirano na ciljnoj frekvenciji od 100MHz (Tclk = 10ns).

Ukupno simulirano vreme	13896780100 (ns)
Ukupno čitanja	3127968008
Ukupno upisa	37536000
Ukupno transakcija	3165504008
Ukupan protok podataka	12075.4 MB

Tabela 3. Estimacija performansi

Na osnovu inicijalnih performansi memorije, model memorije je unapređen da simulira **AXI Burst** mod i pristupstvo lokalnih **BRAM** bafera. Latency po podatku je smanjen na 10 ns kako bi simulirao kontinualni protok podataka.

-Rezultat je drastično smanjen na 13.89 sekundi (pre AXI Burst-a je bilo oko 70 sekundi). Ova optimizacija nam dokazuje da je hardverska implementacija pomoću Burst mod-a neophodna za postizanje ciljnih performansi.

-Rezultati dobijeni na virtuelnoj platformi upoređeni su sa procenama iz Vitis HLS alata radi verifikacije tačnosti modela.

Metoda	Procenjeno Vreme	Odstupanje
Vitis HLS	~17.00s	-
Virtuelna Platforma (VP)	13.89s	~18%

Tabela 4. Razlika u vremenu izvršavanja između dve metode

-Odstupanje od ~18% je unutar prihvatljivih granica za LT modelovanje i ukazuje na to da VP model optimističnije predviđa efikasnost magistrale.

Implementacija hardvera (HLS)

-Sinteza na viskom nivou (HLS) omogućava konverziju algoritma napisanog u C++ jeziku u jezik za opis hardvera (VHDL/Verilog), čime se drastično ubrzava process projektovanja.

-Ključne optimizacije dizajna:

- **Protočna obrada (eng. Pipelining):** Korišćenjem **#pragma HLS PIPELINE II=1**, postignuto je da hardver obrađuje nove podatke u svakom taktu, bez da čeka završetak prethodne operacije.
- **Memorijsko particionisanje:** Kako bi se omogućio paralelni pristup podacima, ulazni podaci su particionisani pomoću **#pragma HLS ARRAY_PARTITION**. Ovo omogućava da se u jednom taktu pročita više elemenata istovremeno, čime se eliminišu uska grla (eng. bottleneck) u pristupu memoriji.
- **Tiling tehnika:** Zbog ograničenih resursa na FPGA čipu (konkretno BRAM memorija), velike matrice (1500 x 512) se ne učitavaju odjednom. Umesto toga, primenjuje se tehnika deljenja na manje blokove (eng. tiles), gde hardver učitava i obrađuje deo po deo matrice, čime se optimizuje upotreba lokalne memorije.
- **Kvantizacija:** radi dodatnog ubrzanja i smanjenja potrošnje memorije, može se izvršiti konverzija tipova podataka, uz minimalan gubitak tačnosti modela.

-Kako bi se hardverski modul integrisao u SoC (eng. System-on-Chip) arhitekturu, definisani su sledeći komunikacioni protokoli putem **#pragma HLS INTERFACE**. Sistem koristi **AXI4-Master** protokol za prenos velikih količina podataka (matrica), a **AXI4-Lite** za kontrolne signale.

Port	Tip interfejsa	Opis i uloga u sistemu
Q, K, V	m_axi	Glavni memorijski interfejs za čitanje ulaznih matrica iz DDR memorije.
W	m_axi	Interfejs za učitavanje matrice težina za finalnu projekciju (512 x 512)
b	m_axi	Interfejs za učitavanje bias vektora (1 x 512)
Y	m_axi	Izlazni interfejs putem kog se upisuju konačni izačunati podaci nazad u memoriju.
return	s_axilite	Kontrolni registar kojim CPU upravlja

Tabela 5: Mapiranje portova modula

-Zbog ograničenih resursa na FPGA čipu i veličine matrica (1500 x 512), implementiran je sekvencijalni pristup obrade podataka. Algoritam izvršava sledeće korake za svaku ulaznu sekvencu:

- Učitavamo jedan red matrice Q u lokalnu keš memoriju.
- Matrica K je prevelika da bi stala cela na čip. Zato se učitava u manjim blokovima, npr. 64 reda po bloku.
- Vrš se množenje vektora Q sa trenutnim blokom K^T . Rezultati se akumuliraju kako bi se dobili parcijalni skorovi.
- Nakon što su obrađeni svi blokovi K matrice, na dobijeni niz skorova primenjuje se softmax funkcija kako bi se dobile verovatnoće.
- Izračunate verovatnoće se množe sa matricom V koristeći Tiling pristup, kako bi se dobio vektor.
- Dobijeni vektor se množi sa izlaznom matricom težina W_{out} i daje se bias b_{out} .
- Konačan rezultat se upisuje u DDR memoriju preko Y porta.

-Ovakva arhitektura omogućava obradu dugačkih sekvenci uz fiksnu potrošnju resursa na čipu.

HLS Koncept	Funkcija/petlja u kodu	Opis implementacije
Lokalno keširanje	<code>load_q_row(...)</code>	Učitava jedan red matrice Q u lokalni niz (<code>Q_local</code>) kako bi se izbegao pristup DDR-u u svakoj iteraciji.
Tiling	<code>LOOP_TILES</code> (u <code>compute_heads</code>)	Glavna petlja koja deli veliku matricu K na manje blokove (<code>TITLE_ROWS</code>) radi uštede BRAM-a
Paralelizacija	<code>#pragma HLS ARRAY_PARTITION</code>	Deli memorijske blokove na delove kako bi hardver mogao da čita te delove u istom taktu
Dataflow	<code>#pragma HLS DATAFLOW</code>	Omogućava da se čitanje podataka i obrada vrše istovremeno
Pipelining (protočnost)	<code>compute_output(...)</code>	Množi izračunate verovatnoće sa matricom V. Koristi <code>II=1</code> direktivu da bi generisao jedan rezultat u svakom taktu

Tabela 6: Ključne funkcije HLS implementacije

-Jedan od ključnih zahteva za obradu govora u realnom vremenu je brzina. U funkciji **compute_output**, koja množi vektor verovatnoće sa matricom V, primenjena je tehnika protočne obrade (pipelining).

-Sledeći isečak koga prikazuje kako se ova optimizacija postiže korišćenjem pragma direktiva:

```
void compute_output(const data_t *V, soft_t probs_buffer[NUM_HEADS][ROWS_W],
```

```

        data_t output_row[COLS_W]) {
#pragma HLS INLINE off
data_t V_row_local[COLS_W];
#pragma HLS ARRAY_PARTITION variable=V_row_local cyclic factor=4

for(int i=0; i<COLS_W; i++) {
    #pragma HLS PIPELINE II=1
    output_row[i] = 0;
}

PROJ_LOOP: for(int k = 0; k < ROWS_W; k++) {

    LOAD_V: for(int j = 0; j < COLS_W; j++) {
        #pragma HLS PIPELINE II=1
        V_row_local[j] = V[k * COLS_W + j];
    }

    UPDATE_ALL_HEADS: for(int h = 0; h < NUM_HEADS; h++) {
        #pragma HLS UNROLL
        soft_t prob = probs_buffer[h][k];
        int base_idx = h * HEAD_COLS;

        UPDATE_HEAD_VEC: for(int j = 0; j < HEAD_COLS; j++) {
            #pragma HLS PIPELINE II=1
            #pragma HLS UNROLL factor=4
            output_row[base_idx + j] += (data_t)((acc_t)prob *
            (acc_t)V_row_local[base_idx + j]);
        }
    }
}
}
}

```

- Protočna obrada **#pragma HLS PIPELINE II=1** je najvažnija instrukcija u HLS dizajnu. Parametar II=1 kaže hardveru da ne čeka da se trenutna operacija množenja završi pre nego što započne sledeću. Umesto toga, hardver formira “traku” gde u svakom taktu izlazi jedan gotov rezultat.
- Bez ove linije, obrada bi trajala hiljadama ciklusa. Sa njom, hardver dostiže maksimalnu teoretsku propusnu moć.

-Nakon izvršene sinteze hardverskog dizajna za ciljnu FPGA platformu, dobijeni su rezultati koji pokazuju balans između brzine izvršavanja i potrošnje. Tabela 5 prikazuje finalne rezultate sinteze.

Modul	Latency		BRAM	DSP	FF	LUT
multi_head_attention	1647522725 (cycles)	1.706e+10 (ns)	233 (83%)	136 (61%)	40986 (38%)	40526 (76%)

Tabela 7. Performanse arhitekture

-Ukupno kašnjenje najviše zavisi od količine podataka koje trebamo da učitamo i brzine čitanja. Zavaljujući Dataflow arhitekturi, dok se jedan blok podataka učitava, drugi se već obrađuje.

-S obzirom na to da koristimo fixed point tipove podataka, VITIS HLS koristi **DSP** jedinice za množenje, čime se štedi **LUT** logika za kontrolne signale.

-Iako **K_tile** zauzima memoriju, to je neophodan kompromis kako bi se izbegao pristup sporoj eksternoj memoriji za svaku operaciju množenja.